

How a Shell Works

Project: Tiny Shell (myShell)
Author : Pham Van Sam
ID student : 202400114
Course: Operating Systems
University: Hanoi University of Science and Technology

Contents

1	What is a Shell?	2
2	The Main Loop — Heart of Every Shell	2
3	Parsing — How the Shell Understands You	3
3.1	Parsing Algorithm	3
4	Executing Commands — CreateProcess() in Depth	4
4.1	What is CreateProcess()?	4
4.2	The Process Creation Timeline	4
4.3	The PROCESS_INFORMATION Structure	5
5	Foreground vs. Background Execution	5
5.1	Side-by-Side Code Comparison	6
6	Background Process Management	6
6.1	Data Structure	6
6.2	Management Commands and Their APIs	7
7	Built-in Commands — Executed Inside the Shell	7
7.1	Why Must These Be Built-in?	7
8	Handling CTRL+C — Graceful Signal Management	8
8.1	The Problem	8
8.2	The Solution	8
8.3	Implementation	8
9	Executing Batch Files (.bat)	9
10	Complete Execution Flow	9
A	Windows API Quick Reference	11

1 What is a Shell?

A shell is a **command-line interface** (CLI) program that acts as an **intermediary** between the user and the operating system. When you type a command such as `notepad hello.txt`, the shell performs the following sequence of operations:

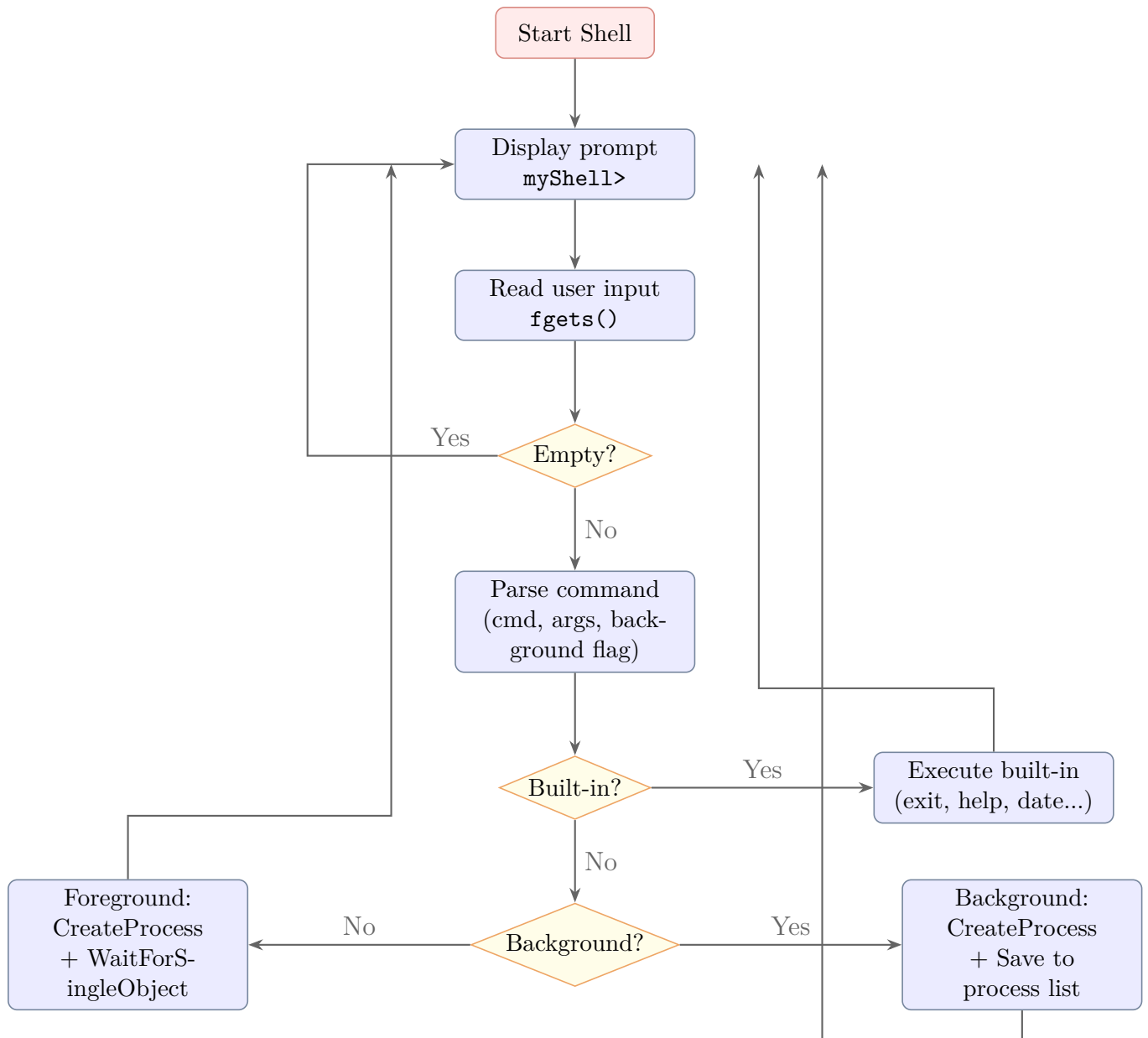
1. **Read** the string of characters you entered.
2. **Parse** it to determine which program to run and with what arguments.
3. **Request** the OS to create a new child process to execute that program.
4. **Wait** (or not) for the child process to terminate.
5. **Return** to the prompt and wait for the next command.

In essence, a shell is nothing more than an infinite loop that reads, evaluates, and prints — commonly referred to as a **REPL** (Read-Eval-Print Loop).

2 The Main Loop — Heart of Every Shell

Important

The **REPL loop** is the single most important concept. Every shell, from `cmd.exe` to `bash` to our `myShell`, is built around this loop. If you understand this, you understand the entire program.



3 Parsing — How the Shell Understands You

When the user types: `notepad hello.txt &`

The parser must decompose this into three distinct pieces of information:

Component	Value	Meaning
Command	<code>notepad</code>	The program to execute
Arguments	<code>hello.txt</code>	Parameters passed to the program
Background flag	<code>&</code>	Run concurrently with the shell

3.1 Parsing Algorithm

1. Read the entire input line
2. Strip the trailing newline `'\n'`

```
3 3. Check if the last non-space character is '&'
4   -> If yes: set isBackground = true, remove '&'
5   -> If no: set isBackground = false
6 4. Tokenize by spaces using strtok()
7   -> First token = command name
8   -> Remaining tokens = arguments
9 5. Concatenate into a full command line for CreateProcess()
```

Note

Unlike Unix, where `execvp()` takes an array of arguments, the Windows API `CreateProcess()` expects the **entire command line as a single string** (e.g., "notepad hello.txt"). However, you still need to extract the command name separately to check whether it is a built-in command.

4 Executing Commands — `CreateProcess()` in Depth

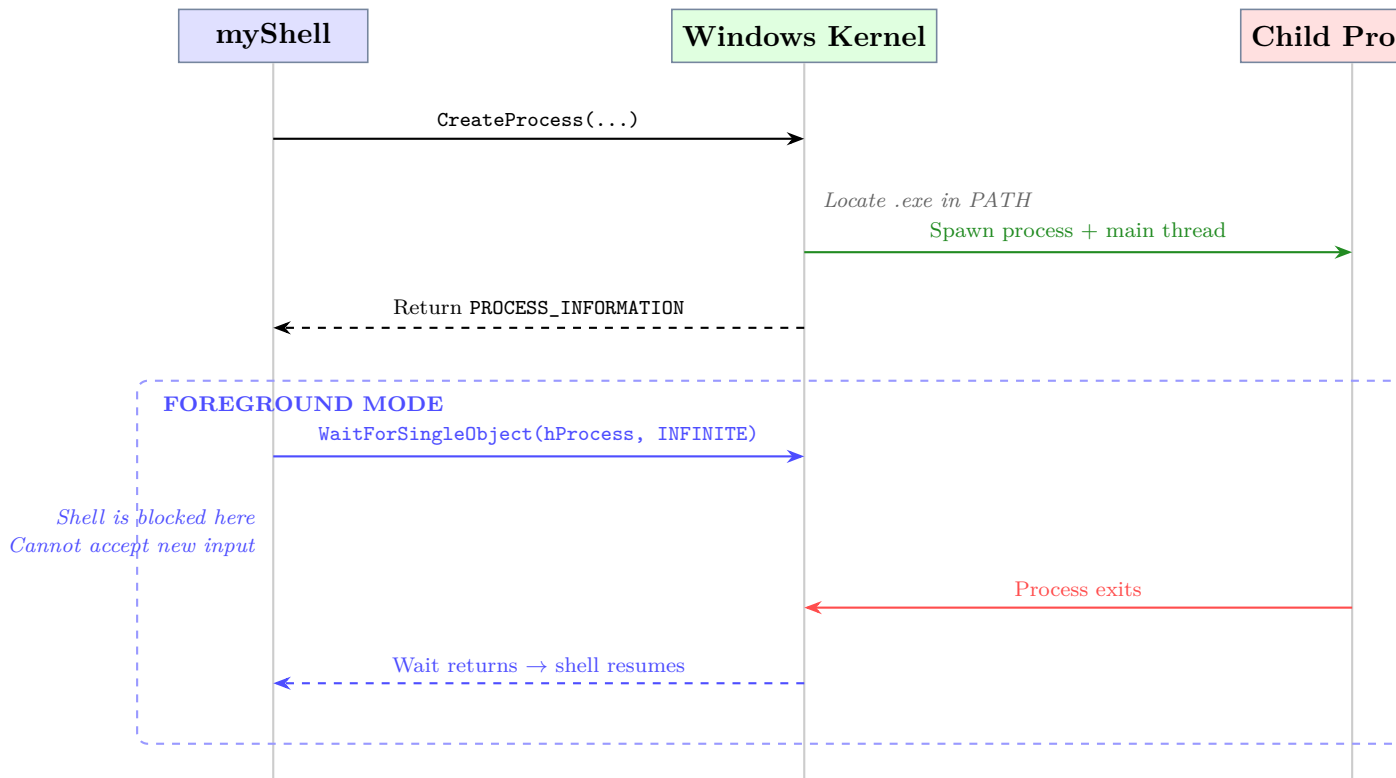
4.1 What is `CreateProcess()`?

`CreateProcess()` is the fundamental Windows API for **spawning a new child process**. Its simplified signature:

```
1 BOOL CreateProcess(
2     LPCSTR lpApplicationName, // Path to .exe (can be NULL)
3     LPSTR lpCommandLine,      // Full command line string
4     ...                        // Security, inheritance, etc.
5     BOOL bInheritHandles,      // Inherit parent handles?
6     DWORD dwCreationFlags,     // Flags (0, CREATE_SUSPENDED, ...)
7     ...
8     LPSTARTUPINFO lpStartupInfo, // Startup configuration
9     LPPROCESS_INFORMATION lpProcessInfo // [OUT] PID, handles
10 );
```

4.2 The Process Creation Timeline

The following diagram illustrates the sequence of events when the shell creates and manages a foreground process:



4.3 The `PROCESS_INFORMATION` Structure

Upon success, `CreateProcess()` populates this output structure:

```

1 typedef struct {
2     HANDLE hProcess; // Handle to the child process (for Wait, Kill)
3     HANDLE hThread;  // Handle to the main thread (for Suspend/Resume)
4     DWORD  dwProcessId; // Process ID (e.g., 12345)
5     DWORD  dwThreadId;  // Thread ID of the main thread
6 } PROCESS_INFORMATION;
  
```

Warning

You **must** store both `hProcess` and `hThread` for background processes.

- `hProcess` is used for `TerminateProcess()` and `WaitForSingleObject()`.
- `hThread` is used for `SuspendThread()` and `ResumeThread()`.

Forgetting to call `CloseHandle()` when a process terminates causes a **handle leak** (similar to a memory leak).

5 Foreground vs. Background Execution

The only difference between the two modes is a single line of code: `WaitForSingleObject()`. Everything else is identical.

	Foreground	Background
Syntax	notepad hello.txt	notepad hello.txt &
Shell blocked?	Yes — waits via WaitForSingleObject()	No — returns to prompt immediately
User experience	Cannot type until process finishes	Can type new commands right away
Handle lifecycle	CloseHandle() immediately after Wait	Store in list; CloseHandle() on kill or natural exit

5.1 Side-by-Side Code Comparison

```

1 CreateProcess(NULL, commandLine, NULL, NULL, FALSE, 0,
2     NULL, NULL, &si, &pi);
3
4 if (isBackground) {
5     // --- BACKGROUND MODE ---
6     // Do NOT wait. Save process info for later management.
7     addToProcessList(pi, commandName);
8     printf("[%d] %s\n", pi.dwProcessId, commandName);
9     // Do NOT close handles here --- we need them later.
10 } else {
11     // --- FOREGROUND MODE ---
12     // Block the shell until the child process terminates.
13     WaitForSingleObject(pi.hProcess, INFINITE);
14     // Child is done --- clean up handles.
15     CloseHandle(pi.hProcess);
16     CloseHandle(pi.hThread);
17 }

```

6 Background Process Management

6.1 Data Structure

We maintain a simple array of structs to track all background processes:

```

1 #define MAX_BG 100
2
3 typedef struct {
4     DWORD pid;           // Process ID
5     HANDLE hProcess;      // Process handle (for Kill, Wait)
6     HANDLE hThread;       // Thread handle (for Suspend, Resume)
7     char cmdName[256];    // Original command name
8     int status;           // RUNNING | STOPPED | TERMINATED
9 } BackgroundProcess;
10
11 BackgroundProcess bgProcesses[MAX_BG];
12 int bgCount = 0;

```

6.2 Management Commands and Their APIs

Shell Command	Windows API	Description
<code>list</code>	<code>WaitForSingleObject(h, 0)</code>	Iterate through array, print PID, name, and status. Use a zero timeout to poll whether each process is still alive without blocking.
<code>kill <pid></code>	<code>TerminateProcess(h, 0)</code>	Forcefully terminate the process. No cleanup occurs in the child.
<code>stop <pid></code>	<code>SuspendThread(hThread)</code>	Suspend the main thread. The process remains in memory but does not execute.
<code>resume <pid></code>	<code>ResumeThread(hThread)</code>	Resume a previously suspended thread.

Tip

When implementing `list`, call `WaitForSingleObject(hProcess, 0)` for each process. If it returns `WAIT_OBJECT_0`, the process has already terminated naturally. In that case, update its status to `TERMINATED` and call `CloseHandle()` to release the kernel resources.

7 Built-in Commands — Executed Inside the Shell

Built-in commands are handled **directly within the shell process itself**, without spawning a child process.

Command	Implementation
<code>exit</code>	Break the main loop → terminate the shell
<code>help</code>	Print a usage guide listing all available commands
<code>date</code>	Call <code>GetLocalTime()</code> → print the current date
<code>time</code>	Call <code>GetLocalTime()</code> → print the current time
<code>dir</code>	List files via <code>FindFirstFile()</code> + <code>FindNextFile()</code>
<code>path</code>	Call <code>GetEnvironmentVariable("PATH", ...)</code> and print the result
<code>addpath <p></code>	Read the current <code>PATH</code> , append <code>;<p></code> , then call <code>SetEnvironmentVariable("PATH", ...)</code>

7.1 Why Must These Be Built-in?

- `exit` must run inside the shell process. You cannot spawn a child process to terminate the parent — the parent would simply continue running.
- `addpath` modifies the shell's own environment variables. On Windows (and Unix), each process has its own **private copy** of the environment. If `addpath` ran in

a child process, the modification would vanish when the child exits, leaving the parent's PATH unchanged.

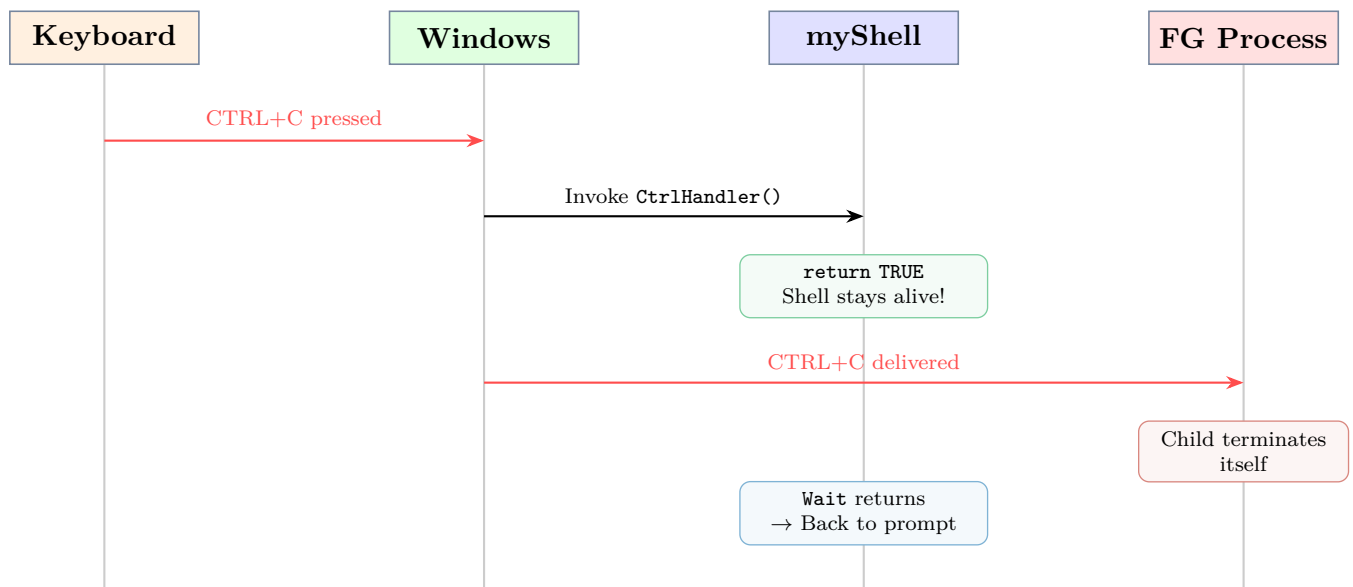
8 Handling CTRL+C — Graceful Signal Management

8.1 The Problem

By default, when the user presses CTRL+C, Windows sends a CTRL_C_EVENT to **all processes** attached to the same console — both the shell and any running child process. Without intervention, the shell itself would terminate.

8.2 The Solution

Register a custom handler using `SetConsoleCtrlHandler()` that intercepts the signal:



8.3 Implementation

```

1 // Global flag: is there a foreground process running?
2 volatile BOOL isRunningForeground = FALSE;
3
4 BOOL WINAPI CtrlHandler(DWORD ctrlType) {
5     if (ctrlType == CTRL_C_EVENT) {
6         if (isRunningForeground) {
7             // The child process will receive CTRL+C from the OS
8             // and terminate itself. We just need to survive.
9             return TRUE; // TRUE = "I handled it, don't kill me"
10        }
11        // No foreground process --- just reprint the prompt.
12        printf("\nmyShell> ");
13        return TRUE;
14    }
15    return FALSE;
16 }
  
```

```
14     }
15     return FALSE; // Let the OS handle other signals
16 }
17
18 // Register once at startup:
19 SetConsoleCtrlHandler(CtrlHandler, TRUE);
```

Important

Returning TRUE from the handler tells Windows: “I have handled this signal; do not terminate my process.” Returning FALSE causes the default behavior, which terminates the shell.

9 Executing Batch Files (.bat)

A .bat file is not an executable — it is a script interpreted by `cmd.exe`. On some Windows versions, calling `CreateProcess()` directly with a .bat file may fail. The reliable solution is to invoke the command interpreter explicitly:

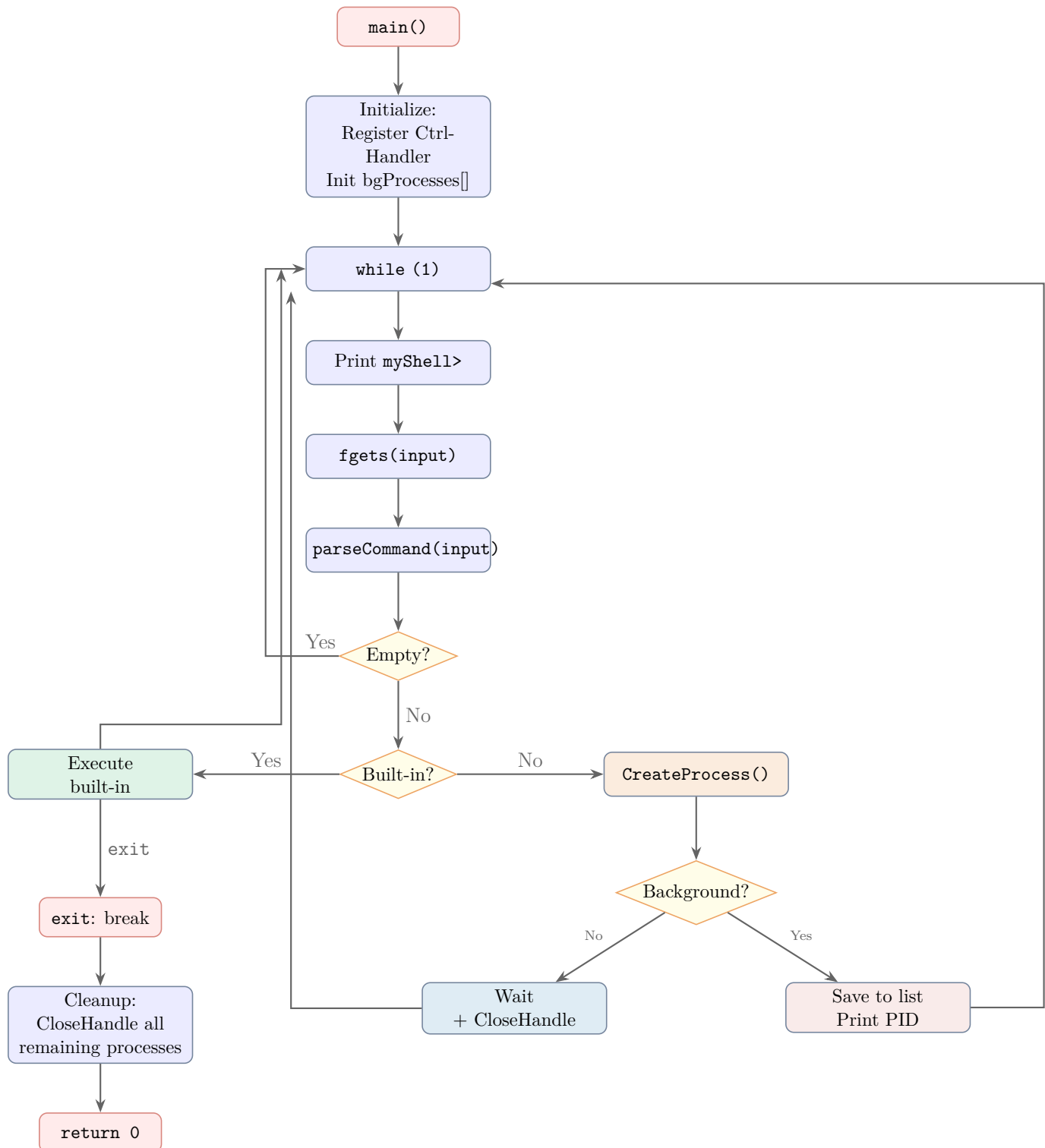
```
1 // When the command ends with ".bat":
2 char fullCmd[1024];
3 sprintf(fullCmd, "cmd.exe /c %s", originalCommandLine);
4 CreateProcess(NULL, fullCmd, NULL, NULL, FALSE, 0,
5               NULL, NULL, &si, &pi);
```

Alternatively, you can **always** pass `NULL` as `lpApplicationName` and let Windows resolve the file type automatically:

```
1 CreateProcess(
2     NULL,           // Let Windows figure out the executable
3     commandLine,    // "script.bat arg1 arg2"
4     NULL, NULL, FALSE, 0, NULL, NULL, &si, &pi
5 );
6 // Windows detects the .bat extension and invokes cmd.exe internally
```

10 Complete Execution Flow

The following diagram summarizes the entire architecture of `myShell`, from initialization to shutdown:



A Windows API Quick Reference

API Function	Purpose in myShell
CreateProcess()	Spawn a new child process from a command line string
WaitForSingleObject()	Block until a process terminates (foreground mode)
TerminateProcess()	Forcefully kill a background process
SuspendThread()	Pause a background process
ResumeThread()	Resume a paused background process
CloseHandle()	Release a kernel handle (process or thread)
SetConsoleCtrlHandler()	Register a custom CTRL+C handler
GetLocalTime()	Retrieve the current system date and time
GetEnvironmentVariable()	Read an environment variable (e.g., PATH)
SetEnvironmentVariable()	Modify an environment variable
FindFirstFile()	Begin directory enumeration
FindNextFile()	Continue directory enumeration
